

FrontierAtlas: AI Data Intelligence Pipeline Architecture

Author: Yash | System Specification: Version 1.0 | Project: GraphOne / FrontierAtlas Demo Task

1. Executive System Overview

FrontierAtlas is an end-to-end AI ecosystem intelligence pipeline designed to collect, extract, normalize, and structure information from multiple public sources. The current system covers five major verticals: AI startups, AI products, research papers, jobs, and news.

The architecture separates data acquisition, LLM-based extraction, entity resolution, freshness filtering, and output delivery so that individual components can be improved or scaled independently.

2. Core Ingestion Pipeline

Layer	Components	Responsibility
1. Acquisition	aiohttp, Requests, RSS/Feedparser, public APIs	Collect startup, product, paper, job, and news data from multiple external sources.
2. Extraction	Groq API, LLM structured extraction	Convert semi-structured or unstructured source content into consistent JSON-style fields.
3. Resolution	RapidFuzz, normalization, canonical seed list	Resolve duplicate or differently formatted company names against canonical entities.
4. Validation	Dateparser, freshness rules, JSON validation	Remove stale signals and validate extracted records before they reach the final dataset.
5. Delivery	Pandas, CSV, Google Sheets	Produce structured datasets and a reviewer-friendly six-tab data hub.

3. Data Sources

Vertical	Primary Source	Purpose
Startups	Y Combinator public company data	Startup/company information and URLs.
Products	AI-Tools-List	AI tools and product descriptions.
Research	Semantic Scholar	Research paper metadata and abstracts.
News	AI-focused RSS feeds	Recent AI ecosystem news signals.
Jobs	RSS feeds + RemoteOK	AI, ML, Data Science, and related roles.

4. 24-Hour Freshness Filtering

News and job signals are time-sensitive, so FrontierAtlas applies a strict rolling 24-hour freshness rule.

- **Date normalization:** dateparser converts relative values such as '2 hours ago' into standardized datetime values.
- **UTC normalization:** timestamps are converted into a common UTC representation where source information permits.
- **Hard cutoff:** records older than the configured 24-hour window are excluded from the current signal dataset.
- **Deduplication:** source URLs can be used as deterministic record identifiers to avoid processing the same signal repeatedly.

5. Resilience and Failure Handling

External APIs and web sources can fail temporarily. FrontierAtlas therefore treats failures as expected conditions rather than assuming every request will succeed.

HTTP 429 — Rate Limit Handling

- External calls use retry logic with exponential backoff and jitter where supported.
- Async concurrency is controlled using `asyncio.Semaphore` so the pipeline does not intentionally create an uncontrolled request burst.
- The LLM extraction layer supports fallback execution when the primary model request fails or returns unusable output.

HTTP 413 — Payload / Context Protection

Large text fields are truncated before LLM processing where the full content is not required for the target extraction. For example, product descriptions can be restricted to a fixed character budget.

6. Multi-Tier LLM Extraction Architecture

The LLM layer converts unstructured descriptions into structured fields. The design avoids making the entire pipeline dependent on one successful model response.

Tier	Role	Failure Handling
Primary Model	Fast structured extraction	Retry on temporary failure.
Fallback Model	Alternative extraction path	Used when the primary request fails or output is invalid.
Safe Default	Protect downstream processing	Used for fields where a safe default is defined.

7. Entity Resolution Strategy

Web data frequently contains multiple representations of the same company. FrontierAtlas uses normalization and fuzzy matching against a canonical seed list to reduce duplicate entities.

- **Normalization:** lowercasing, whitespace cleanup, punctuation normalization, and legal suffix handling.
- **Exact matching:** normalized names are first compared directly against the canonical entity list.
- **Fuzzy matching:** RapidFuzz is used only after normalization with a configured similarity threshold.
- **False-positive protection:** the matching strategy avoids treating a common word such as 'AI' as sufficient evidence that two companies are the same entity.
- **Audit trail:** raw-to-canonical decisions are retained in `entity_mapping_log.csv`.

8. Scaling Strategy for 500,000+ Records

The demo implementation focuses on the required dataset scale, while the architecture is designed so that ingestion and processing can be distributed as volume increases.

Challenge	Current Design	Production Extension
Large Dataset	Pagination and independent scraper modules	Distributed workers processing independent shards
Request Concurrency	<code>asyncio</code> + <code>Semaphore</code>	Worker pools with centralized task queue
Rate Limits	Retry, backoff, jitter, and LLM fallback	Per-provider quotas and adaptive scheduling
Storage	CSV + Google Sheets	PostgreSQL with batch writes and indexing

9. Distributed Production Architecture

At larger scale, the independent scraper modules can be executed as distributed jobs. A task queue such as Redis/SQS with worker orchestration such as Celery or Kubernetes Jobs can distribute pagination ranges, source categories, or other shards.

The key principle is to keep ingestion workers as stateless as possible. Shared state such as processed URLs, checkpoints, and canonical entities should be stored in centralized infrastructure.

10. Production Storage Strategy

Technology	Role	Reason
PostgreSQL	Primary structured database	Suitable for startups, products, papers, jobs, and news with indexed relational fields and JSONB where needed.
Neo4j	Graph relationship layer	Useful for multi-hop relationships such as startup → founder → product → paper.
pgvector / Vector DB	Semantic search	Useful for similarity search across paper abstracts, product descriptions, and startup information.

11. Distributed Deduplication and Freshness

When multiple crawler workers are running, the system needs a shared mechanism to prevent duplicate processing.

- **Deterministic key:** the normalized source URL can act as an idempotency key.
- **Database constraint:** production storage can enforce uniqueness on the source URL or URL hash.
- **Shared state:** Redis or database checkpoints can track last-seen timestamps and processing status across workers.
- **Shard ownership:** workers can be assigned specific feeds, source categories, or pagination ranges to reduce duplicate crawling.

12. Future High-Volume Web Acquisition

For sources that require browser rendering or impose strict anti-automation controls, a production deployment could introduce browser-based workers, controlled request scheduling, proxy infrastructure, and source-specific adapters.

These components are considered production extensions rather than requirements of the current demo implementation.

13. Known Limitations and Trade-offs

- The demo was developed within a limited implementation window and focuses on the required dataset coverage rather than fully processing 500,000 records during the demonstration.
- Public APIs and RSS sources can change, become unavailable, or return incomplete data. Source-specific adapters are therefore required for long-term maintenance.
- Entity resolution uses deterministic normalization and fuzzy matching. Ambiguous entities are safer to leave unmatched than to force an incorrect canonical mapping.
- CSV and Google Sheets are suitable for demonstration and review, but a production deployment would require a database with indexing, constraints, monitoring, and controlled ingestion.

14. Deliverables

- **Google Sheets Data Hub:** Six-tab structured output containing startups, products, papers, jobs, news, and entity mappings.
- **Python Pipeline:** Modular scraper, extraction, resolution, and processing scripts under the `src/` directory.
- **Structured Outputs:** CSV datasets under the `output/` directory.

- **Architecture Documentation:** This PDF describing the current system and production scaling approach.

FrontierAtlas — AI ecosystem data ingestion, extraction, normalization, and intelligence pipeline.